

From Code to Carbon

Sustainable Practices in Software Engineering

Iztok Fister, Jr.

University of Maribor

`iztok@iztok.link`

June 5, 2026



CC BY-NC-SA 4.0

This talk is based on two papers

Paper 1: The opening hook

A Life-Cycle Assessment of the Hidden Environmental Cost of Selfies

WWW '26

Uses the selfie as a measurable micro-action: capture, upload, process, storage, revisiting, deletion.

Paper 2: The software-engineering core

Profiling the Carbon Footprint of Performance Bugs

ACDSA '24

Shows that inefficient code paths can be profiled as extra energy use and extra CO₂e.

Unifying thesis: sustainability starts when invisible digital work becomes visible, measurable, and reducible.

Opening hook: the invisible weight of a selfie

5.4B

social media users worldwide
in 2025

212M

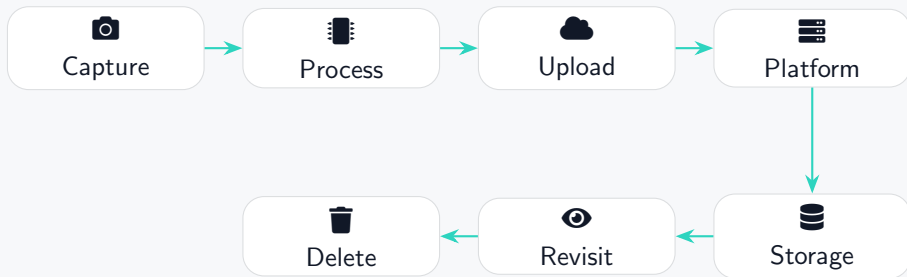
selfies taken every day

54h

per person per year spent
taking selfies

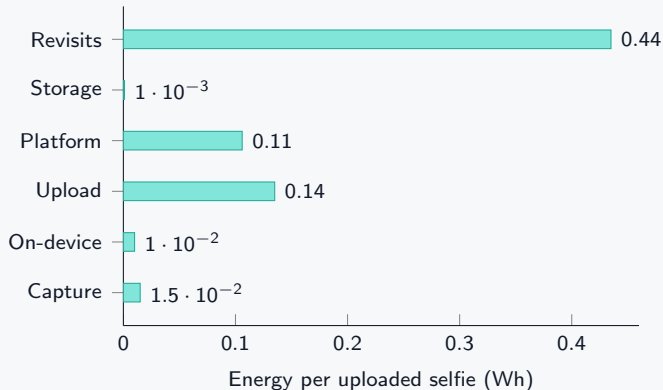
A selfie feels weightless. The motivation paper treats it as a micro-unit of the digital economy: capture, process, upload, store, revisit, and eventually delete. The interesting question is not whether one selfie is big - it is whether billions of tiny actions become visible at planetary scale.

A selfie is not one action - it is a life cycle



The paper motivates a **cradle-to-grave** view aligned with ISO 14067:2018, rather than only counting the upload.

The surprising part: the biggest cost is often after the snap



0.702 Wh

total energy per uploaded selfie
in the paper's estimate

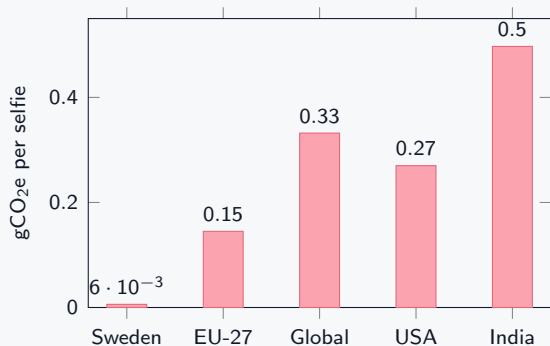
0.332 gCO₂e

using the global average elec-
tricity emission factor

Re-viewing by friends dominates

the estimate: the social life of the
image matters.

Same selfie, different grid: why Sweden is lower than India



Listener hint: Not all kWh are equal. The attached paper uses emission factors: Sweden 0.008 kgCO₂e/kWh; India 0.708 kgCO₂e/kWh. The same Wh of computation therefore becomes radically different CO₂e.

Sweden's grid is much cleaner because it relies heavily on low-carbon electricity such as hydro, nuclear, and wind. India has been improving, but its electricity is still far more coal-intensive. That is why carbon-aware software asks: *where*

The real opening message: awareness, not guilt

Global annual selfies

If uploaded, their footprint would require about

513,875 trees

to offset, roughly

29 Central Parks

Why this belongs at the beginning:

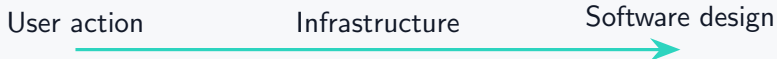
- One selfie is tiny, but it is measurable.
- Billions of routine digital actions accumulate.
- The method makes invisible infrastructure visible.
- The same logic extends to photos, videos, stories, reels, and other shared content.

Transition: If user behavior has a carbon footprint, what about the software that makes it possible?

From selfies to software

The carbon footprint is not only in what users do.

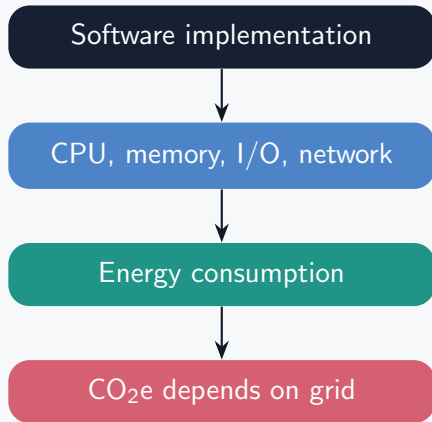
It is also in what our code makes machines do.



Green ICT: the software engineering problem

Green ICT aims to mitigate carbon emissions from ICT production, applications, and services.

Sustainable software engineering therefore cannot stop at hardware efficiency. It must also ask how software choices change execution time, resource use, and energy demand.



Performance bugs: when code wastes carbon

Definition from the paper: performance bugs are unnecessarily inefficient code chunks that prolong execution time and degrade resource utilization.

The key idea is simple: if a bug makes a program run longer or use more resources, it also increases energy use - and, depending on the grid, carbon emissions.

Software bug

wrong result

Performance bug

right result, unnecessary work

Carbon bug

right result, unnecessary emissions

Case study: four ways to do the same work in C++

Performance-bug version: **Vector**

```
for i = 0 to MAX_ITER:  
    vec.pop_back()  
    vec.insert(head, i+1)
```

The vector implementation repeatedly inserts at the head, causing expensive movement/reallocation behavior when used heavily.

Lower-carbon alternative: **List**

```
for i = 0 to MAX_ITER:  
    last->num = i + 100  
    first = last  
    last = last->prev
```

The double linked list changes pointers and values instead of forcing repeated vector-style rearrangement.

The paper compares **Vector**, **Raw**, **Array**, and **List** implementations on a laptop, Raspberry Pi 3, and Apple iPad. Same goal, different energy consequences.

The four implementations: what changed?

Vector: simulated performance bug

Uses `pop_back()` and `insert(head, ...)` repeatedly. This keeps the program correct, but creates costly data movement when the operation is repeated many times.

Array: fixed C++ array

Uses `long int vec[100]` and the same manual shift idea. It removes vector abstraction overhead, but still moves values sequentially.

Raw: vector without high-level insertion

Still stores data in vector, but manually shifts elements: `vec[j] = vec[j-1]`. It avoids `insert`, yet performs repeated element assignments inside the loop.

List: double linked list

Changes the structure of the work: update one node value and rotate `first/last` pointers. Less repeated movement, lower measured footprint in this experiment.

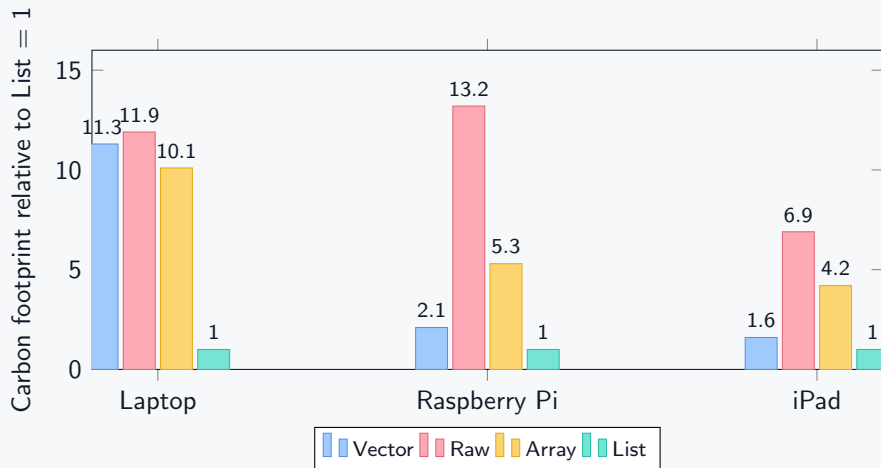
Same final sequence, different amount of machine work. That is the software-engineering lesson.

The four implementations: comparison table

Algorithm	Data structure	How it adds a new element at the beginning	Main characteristic
Vector	<code>std::vector</code>	With <code>insert</code>	Most direct, but can be inefficient
Raw	<code>std::vector</code>	Manual shifting of elements	Avoids <code>insert</code> , but still uses vector
Array	Plain array	Manual shifting of elements	Less abstraction, often faster than vector-based solutions
List	Doubly linked list	Pointer updates	Least data movement, most efficient in the tests

The comparison shows that all four implementations preserve the same logical behavior, but they impose very different amounts of data movement on the machine.

Result: the “right” data structure can be an emissions decision



Across all three platforms, the **List** implementation was the greenest in the experiment. The Raw version was often the worst, especially on Raspberry Pi and iPad.

Why the List implementation wins in this benchmark

List changes the operation

```
for i = 0 to MAX_ITER:  
    last->num = i + 100  
    first = last  
    last = last->prev
```

No repeated shifting of all 100 elements; the program mostly changes pointers and one value.

Sustainable-software interpretation

- Reduce unnecessary data movement.
- Treat data structure choice as an energy decision.
- Validate with profiling because cache behavior and hardware can change the winner.

The paper does not claim “lists are always greener.” It shows a stronger principle: the greenest code is often the code that forces the machine to do the least unnecessary work.

Two profiling lenses: software vs. hardware measurement

Software method: powertop

- Linux laptop, battery-powered measurement.
- Reports discharge rate and activity from CPU, devices, timers, kernel, and interrupts.
- Needs calibration and a comparable battery state before each run.
- Good for quick software-side profiling, but affected by battery non-linearity and reporting intervals.

Hardware method: AVHzY CT-3 power meter

- Measures active electrical power directly through the power path.
- Used for Raspberry Pi 3 and Apple iPad experiments.
- Captures small power changes externally, rather than inferring them from the operating system.
- The paper found hardware metering more accurate for this profiling setup.

Protocol: measure an inactive baseline, run the algorithm under strain, measure an inactive baseline, then subtract the baseline to isolate code-induced energy and CO₂e.

Sustainable practice I: write code that does less work

Start with a simple question

Can the computer get the same answer with fewer steps?

- Choose a data structure that fits the job.
- Avoid moving the same data again and again.
- Avoid loops that repeat unnecessary work.
- Reuse results when it is safe to reuse them.

Correct code is not enough.

It should also avoid wasting time, memory, and energy.

Example from the benchmark: a list can add an item at the front by changing pointers, while a vector may need to move many values.

Sustainable practice II: measure before you guess

Do not only trust intuition

- Fast-looking code is not always the lowest-energy code.
- The best choice can change on a laptop, phone, or small computer.
- Measure the real program on the real device when possible.

Simple test: run the same task, compare time and power, then keep the cleaner version.

What should we watch?

- How long the program runs.
- How much CPU and memory it uses.
- How much data it sends or stores.
- Whether a new change makes things worse.

Main idea: small engineering choices can become large energy savings when software runs many times.

New concern: AI-assisted programming has two footprints

1. Footprint of using the LLM

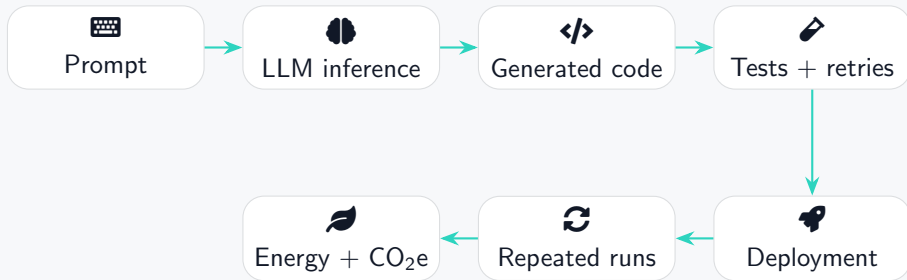
Every request activates data-center computation: model inference, networking, cooling, storage, and sometimes repeated retries until the answer looks right.

2. Footprint of the generated code

The code may then run thousands or millions of times. If it is inefficient, the small LLM cost can become a long-term software carbon cost.

Key message: for programming tasks, sustainability is not only “how expensive was the prompt?” It is also “how much work will this generated code force machines to do later?”

The AI coding pipeline: carbon can enter at every step



Recent green-code studies therefore evaluate both sides: the **energy of AI code generation** and the **energy efficiency of the produced program**.

How green is LLM-proposed source code?

What early empirical studies suggest

- LLMs often generate functionally correct code, but not necessarily energy-efficient code.
- Generated solutions can use weaker algorithms, unnecessary loops, excessive memory, or verbose library calls.
- A “green” answer may require explicit constraints: time complexity, data movement, memory use, and profiling.

Sustainability varies by model, prompt, task, and language. Measure it.

Correct
passes tests

Efficient
fewer operations

Green
less energy on real hardware

Green prompting: ask the model to optimize for sustainability

Weak prompt

```
Write a Python function to solve this problem.
```

May return a correct but wasteful solution: brute force, repeated work, high memory use, or unnecessary dependencies.

Greener prompt

```
Write a solution that minimizes time complexity, data movement, memory allocation, and I/O. Explain the complexity and suggest how to profile energy.
```

Makes efficiency part of the specification, not an afterthought. The result still needs testing and measurement.

Prompting can reduce waste, but it is not a substitute for profiling. The sustainable workflow is: generate, test, measure, refactor, and compare against a baseline.

Takeaway for teams adopting LLM programming assistants

Use LLMs, but add guardrails

- Ask for complexity and trade-off explanation.
- Ask for a low-memory or low-I/O alternative.
- Reject solutions that only “look elegant.”
- Benchmark generated code against a baseline.

Make green review part of code review

- Check hot paths, loops, queries, and data movement.
- Measure runtime, memory, and energy when important.
- Prefer smaller models for simple tasks when acceptable.
- Deploy carbon-aware when computation is flexible.

Final LLM lesson: AI can help us write code faster, but engineers remain responsible for how much energy that code consumes.

The bridge between the two papers

Selfie paper

One everyday digital action has a measurable life-cycle carbon footprint.

same logic →

Code paper

Inefficient implementations can measurably increase energy and carbon.

Main thesis: make invisible energy visible, measurable, and reducible.

From code to carbon

The question is no longer only:

“Does it work?”

The sustainable question is:

“How much work did it force the world to do?”

Measure. Profile. Refactor. Deploy carbon-aware.